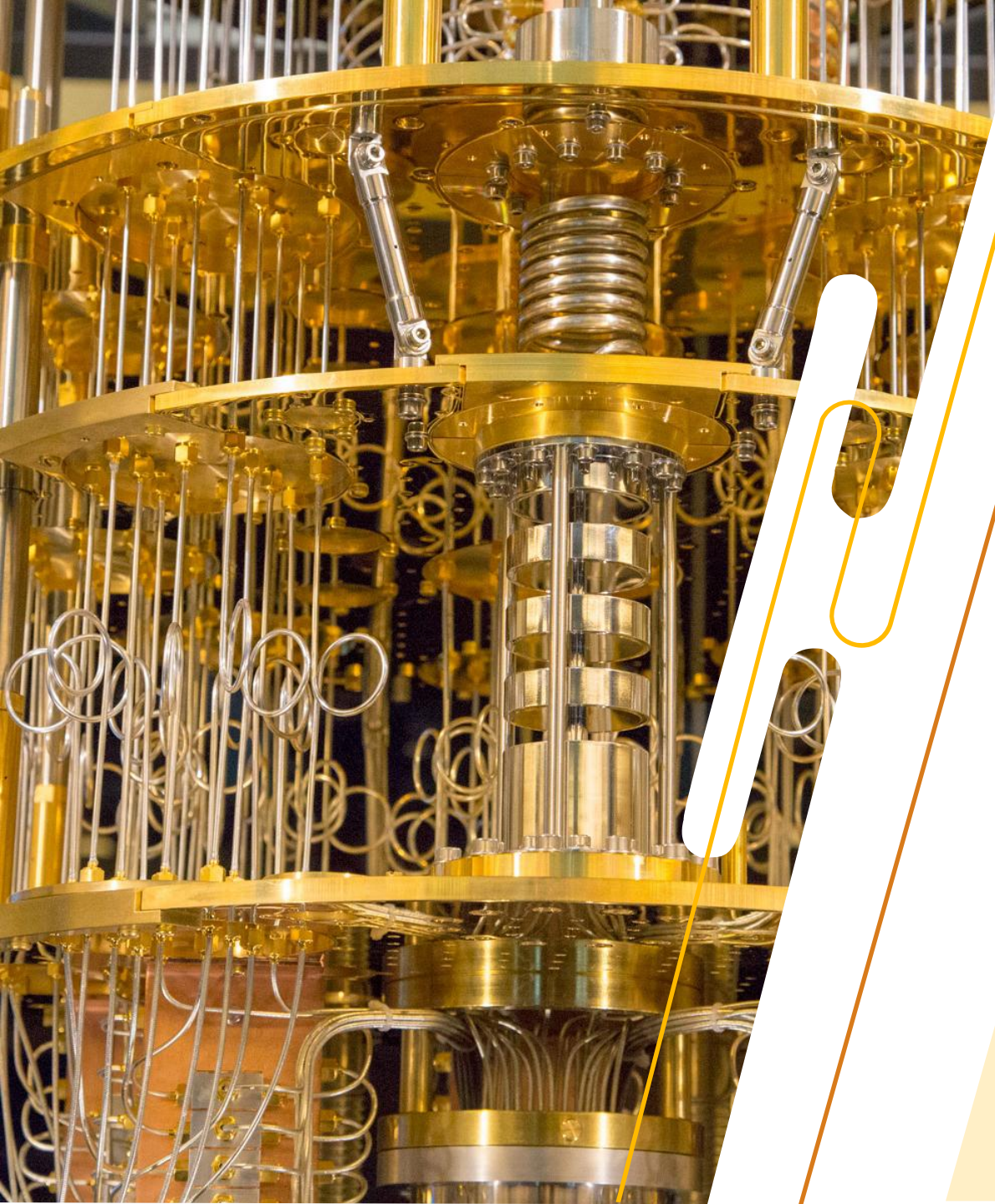




CAPSTONE PROJECT

(OPTIMIZING PARAMETERIZED QUANTUM
CIRCUIT TO INCREASE FIDELITY
BETWEEN TARGET AND OUTPUT STATE)

MENTORED BY:
MAYUKH SARKAR

PRESENTATION BY:
ANMOL RISHIKET
PRAVEEN KUMAR PANDEY
ARYAN KUMAR

Content

- o1. Intro to Quantum computation
- o2. Problem Statement
- o3. Project Overview
- o4. Architecture
- o5. Implementation & References





Introduction

What is Quantum Computation?

Quantum computing is a revolutionary computing paradigm that leverages the principles of quantum mechanics to process information in ways that classical computers cannot. It enables faster computations for specific problems like cryptography, optimization, and simulations.

Qbits

A qubit is the fundamental unit of quantum information, similar to a classical bit but with unique properties:

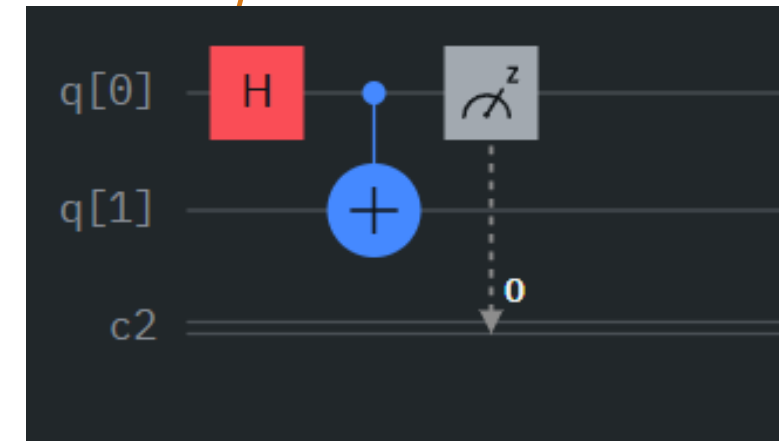
- ❑ **Superposition:** A qubit can exist in both 0 and 1 states simultaneously.
- ❑ **Entanglement:** Qubits can be correlated, influencing each other instantly, regardless of distance.
- ❑ **Interference:** Quantum states can interfere to amplify correct outcomes and cancel incorrect ones.

Quantum Circuits

Quantum circuits function similarly to classical circuits but operate on **qubits** instead of classical bits. Unlike classical bits, which can be either **0** or **1**, qubits can exist in **superposition**, meaning they can be **0**, **1**, or **both simultaneously** due to quantum mechanics.

State vectors

A **state vector** represents the complete quantum state of a system. It is a **complex-valued vector** that encodes the probabilities of different measurement outcomes. For a system with **n qubits**, the state vector has **2^n complex components**, defining all possible states the system can collapse into upon measurement.



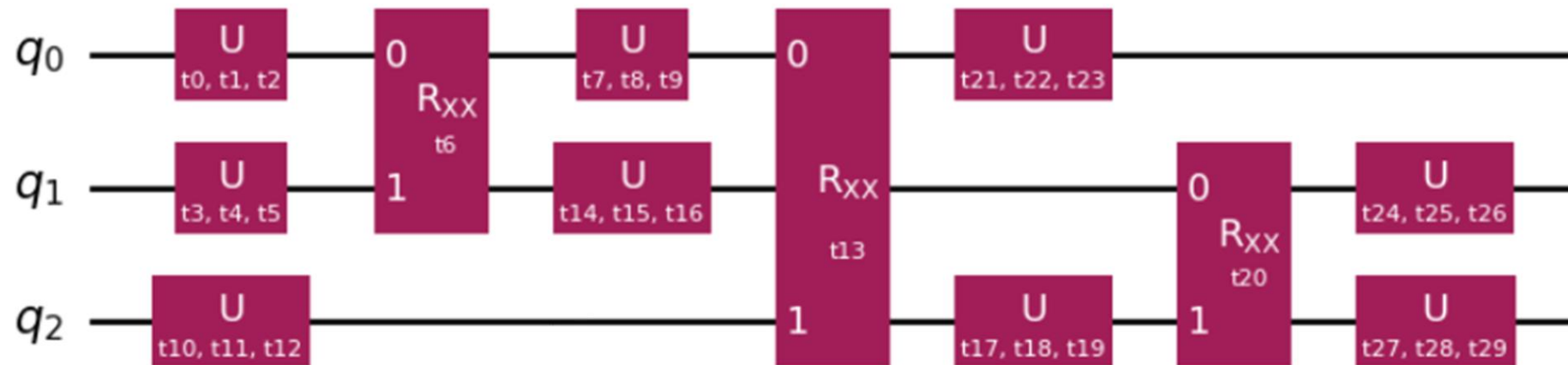
Output state

[$0+0j$, $0+0j$, $0+0j$, $1+0j$]

Quantum Gates

Quantum circuits are built using **quantum gates**, which manipulate qubits to perform computations. Many quantum gates are **parameterized**, meaning they have tunable parameters (e.g., **rotation angles**) that influence how the qubits evolve through the circuit

The parameterized quantum circuit we are working on



Gates

U Gate (General Single-Qubit Rotation)

- **Parameters:** θ, ϕ, λ (e.g., t_0, t_1, t_2)
- **Function:** A universal single-qubit unitary gate that can perform any rotation in the Bloch sphere.
- **Effect:** Controls how a qubit evolves, allowing arbitrary transformations.

RXX Gate (Two-Qubit Rotation around XX Axis)

- **Parameter:** Rotation angle θ (e.g., t_6, t_{13}, t_{20})
- **Function:** A two-qubit gate that applies an **XX interaction**, which is useful in entangling qubits.
- **Effect:** Creates quantum correlations (entanglement) between the connected qubits.

Quantum Simulation Backend

Quantum Frameworks Used:

Fidelity Calculation:

1. Quantum Circuit Execution:

- The circuit is assigned a set of parameters.
- The simulator runs the circuit using **Qiskit Aer** to obtain the output statevector $|\psi_{\theta}\rangle$.

2. Fitness Function:

- Fidelity is computed as: $F = |\langle \psi_{\text{target}} | \psi_{\theta} \rangle|^2$
- In the code, it is calculated using `quantum_info.state_fidelity(target, result)`
- **Backend:** `statevector_simulator` from **Qiskit Aer** is used to simulate quantum states efficiently.
- **Qiskit (IBM):** Open-source quantum computing framework for designing and running quantum circuits.
- **Qiskit Aer:** A high-performance simulator framework that provides backends for statevector and density matrix simulations.
- **Quantum Circuit:** Defined using `QuantumCircuit`, with parameterized quantum gates (`Parameter`).

Problem Statement

Goal

Given a target quantum statevector $|\psi\rangle$, find the optimal parameters θ for a parameterized quantum circuit that generates an output statevector $|\psi'\rangle$ as close as possible to the target.

Fidelity as a Measure of Closeness

- Fidelity (**F**) quantifies the similarity between $|\psi\rangle$ and $|\psi'\rangle$.
- It ranges from **0 to 1**, where **F = 1** indicates a perfect match.

Challenges

- ⚡ **High-Dimensional Parameter Space** – Even small quantum circuits can involve dozens or hundreds of parameters, making optimization complex.
- ⚡ **Barren Plateaus** – Large regions of the optimization landscape have extremely small gradients, making gradient-based methods inefficient.
- ⚡ **Local Optima** – The fidelity function is non-convex, leading to multiple local optima that can trap optimization algorithms.
- ⚡ **Noisy Evaluation** – Real quantum hardware introduces noise, making fidelity measurements imprecise.

Why Reinforcement Learning(RL)?

❖ Global Optimization

RL algorithms, particularly policy gradient methods like **Proximal Policy Optimization (PPO)**, excel at exploring complex, non-convex optimization landscapes.

❖ Noise Robustness

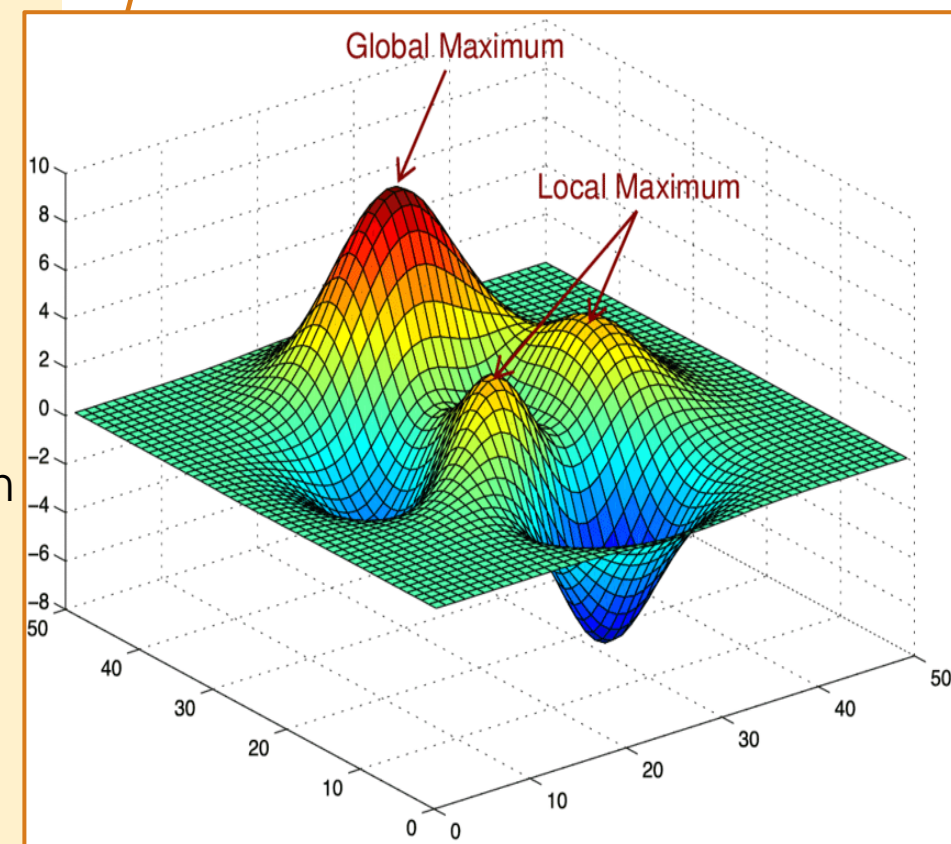
RL can be designed to be **resilient to noise**, making it well-suited for real quantum hardware where measurements are inherently noisy.

❖ Model-Free Learning

- PPO does **not require a differentiable model** of the quantum circuit, unlike traditional gradient-based methods.
- This is crucial for real quantum systems, where exact circuit dynamics may be unknown.

❖ Continuous Control

PPO is inherently designed for **continuous action spaces**, making it a natural fit for optimizing quantum circuit parameters.



Reinforcement Learning Basics

Agent and Environment

RL involves an **agent** that interacts with an **environment** to learn optimal behavior through trial and error.

Key Concepts

❖ States (s):

- Represents the current situation of the environment.
- In our case, the state corresponds to the **current circuit parameters (θ)**.

❖ Actions (a):

- Represents the decision made by the agent.
- Here, the action is the **adjustment to the parameters ($\Delta\theta$)**.

❖ Rewards (r):

- A scalar feedback signal indicating how good an action was.
- In our case, **fidelity (F)** serves as the reward, measuring the closeness of the output state vector to the target.

❖ Policy (π):

- A function (typically a neural network) that maps **states to actions**, guiding the agent's behavior.
- We use a **stochastic policy $\pi_\theta(a | s)$** , which provides a probability distribution over possible actions.

❖ Value Function ($V(s)$):

- Estimates the **expected cumulative reward** from a given state when following the current policy.

Goal of RL

- Learn an **optimal policy** that **maximizes the expected cumulative reward**, leading to better parameter optimization and higher fidelity.

Project Overview

Approach

We employ **Proximal Policy Optimization (PPO)**, a state-of-the-art reinforcement learning (RL) algorithm, to optimize the parameters of a quantum circuit for achieving high fidelity with a target quantum state.

Policy Gradient Method

- **PPO** is a **policy gradient algorithm**, meaning it directly updates the policy parameters to maximize expected rewards.

Clipped Surrogate Objective

- PPO uses a **clipped surrogate objective** to prevent overly large and destabilizing updates to the policy.
- The objective function is given by:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)},$$

Project Overview

- $r_t(\theta)$: Probability ratio of the new policy over the old policy.
- $\hat{A}_t$: Advantage estimate, which measures how much better an action is compared to the expected outcome.
- ϵ : Clipping parameter (e.g., 0.2), which limits policy updates to ensure stability.

Key Benefits of PPO

- ◆ **Stability:**
 - Clipping prevents drastic policy changes, leading to smoother and more stable learning.
- ◆ **Sample Efficiency:**
 - PPO is relatively sample-efficient, requiring fewer interactions with the environment compared to other RL methods

Components of the RL-based Quantum Optimization System

❶ PPO Agent

Contains the **policy network** (which decides parameter updates) and the **value network** (which estimates expected future rewards).

❷ Quantum Circuit Simulator

Simulates the **parameterized quantum circuit** using frameworks like **Qiskit, PennyLane, or Cirq** to generate a quantum statevector.

❸ Fidelity Calculator

Computes the **fidelity (F)** between the **output statevector** from the circuit and the **target statevector**, serving as the reward signal for the RL agent.

WORKFLOW

- ❶ Agent receives circuit parameters (θ) as state (s).
- ❷ Agent outputs parameter adjustments ($\Delta\theta$) as action (a).
- ❸ Simulator executes the quantum circuit with updated parameters ($\theta + \Delta\theta$).
- ❹ Fidelity (F) is computed and used as the reward (r).
- ❺ PPO updates policy and value networks using (s, a, r, s'), refining the parameter optimization process.

State Representation

Input to Policy Network

- The **state (s)** represents the current quantum circuit parameters.
- It is simply a vector of parameters:
- $s = \theta = [\theta_1, \theta_2, \dots, \theta_{30}]$ (For a 30-parameter quantum circuit)

Normalization

- To improve neural network training, parameters can be **normalized** to a standard range, such as:
 - **Rotation Angles** $\rightarrow [0, 2\pi]$
 - **Other Parameters** $\rightarrow$ Scaled between $[-1, 1]$ or $[0, 1]$

Action Space

1) Output of the Policy Network

- The **action (a)** is a vector of **adjustments** to the circuit parameters:
- $a = \Delta\theta = [\theta_1, \theta_2, \dots, \theta_{30}]$ (For a 30-parameter quantum circuit)

3) Policy Output: Gaussian Distribution

- The **policy network** does not output actions directly; instead, it **parametrizes a probability distribution** over possible actions.
- A **Gaussian distribution** is commonly used:
 - The **network outputs a mean (μ) and standard deviation (σ)** for each action component $\Delta\theta_i$. Actions are sampled from:
 - $\Delta\theta_i \sim N(\mu_i, \sigma_i)$

2) Continuous Action Space

- Each $\Delta\theta_i$ is a **continuous value**, allowing smooth parameter updates.

4) Action Bounds

- To prevent **excessive parameter changes**, actions can be **clipped** to a reasonable range:
 - Example: $\Delta\theta_i \in [-0.1, 0.1]$
 - This ensures **stability** in training and prevents drastic changes to quantum circuit parameters

Reward Function

❑ Direct Fidelity as Reward

The most straightforward reward function is the **fidelity** between the generated quantum state $|\psi_\theta\rangle$ and the target state $|\psi_{\text{target}}\rangle$:

$$R=F=|\langle\psi_{\text{target}}|\psi_\theta\rangle|^2$$

❑ Advantages

- **Clear and Intuitive:** Directly measures how close the generated state is to the target.
- **Aligned with Optimization Goal:** The RL agent is incentivized to maximize fidelity.

❑ Potential Issues and Solutions

- **Sparse Rewards:** If achieving high fidelity is difficult, the agent may struggle to learn effectively.

✓ **Solution 1 - Reward Shaping:**

Provide small positive rewards for **any improvement** in fidelity, encouraging gradual progress.

✓ **Solution 2 - Curriculum Learning:**

- Start with **simpler target states**, then progressively introduce harder ones to help the agent learn effectively.
- Encourages faster convergence by providing a broader range of rewards.

Policy Network Architecture

Overview

The policy network is a **fully connected neural network** that maps the state (current quantum circuit parameters) to a probability distribution over parameter updates.

Network Structure

◆ Input:

- The state $\mathbf{s}$ is a **vector of 30 parameters** $\theta = [\theta_1, \theta_2, \dots, \theta_{30}]$.

◆ Hidden Layers:

- **2-3 fully connected (dense) layers.**
- **64-128 neurons per layer** (this is a tuneable hyperparameter).
- **Activation functions:**
 - **ReLU** or **Tanh** for hidden layers to introduce non-linearity.

◆ Output Layer:

- The network **does not directly output actions**. Instead, it **parametrizes a probability distribution** over actions.

• For each parameter θ_i , the network outputs:

- A **mean** μ_i (suggested adjustment).
- A **log standard deviation** $\log \sigma_i$ (controls exploration).

• **Total Outputs:** Since there are **30 parameters**, the network outputs **60 values** ($\mu_1, \dots, \mu_{30}$ and $\log \sigma_1, \dots, \log \sigma_{30}$).

◆ Activation Functions in the Output Layer:

- **Mean (μ) output:** No activation (unbounded).
- **Log Standard Deviation ($\log \sigma$) output:**
 - **Softplus** or **exponential** activation to ensure σ remains positive.

Value Network Architecture

Overview

The value network is a **fully connected neural network** that estimates the **value function** $V(s)$, which predicts the **expected cumulative reward** from a given state.

Network Structure

◆ Input:

- The state s is a **vector of 30 parameters**: $s = \theta = [\theta_1, \theta_2, \dots, \theta_{30}]$

◆ Hidden Layers:

- **2-3 fully connected (dense) layers**, similar to the policy network.
- **64-128 neurons per layer** (tunable hyperparameter).
- **Activation functions**:
 - **ReLU** or **Tanh** for non-linearity.

◆ Output Layer:

- Outputs a **single scalar value**: $V(s)$ which represents the **estimated value of the current state**.
- **No activation function** is applied in the output layer, as value predictions are unbounded

Training Process

START

1 Perform K iterations

- The optimization process runs for **K iterations**, refining quantum circuit parameters over time.

2 Initialize N agents, each starting with different parameters

- The **parameter space** serves as the **environment** in RL.
- Each agent is initialized with **different quantum parameters**, allowing exploration of different regions of the parameter space.

3 Run the policy network for T steps

- Each agent follows its **policy** to adjust parameters over **T timesteps** and interacts with its environment.

4 Compute Advantage estimates $\hat{A}_1, \dots, \hat{A}_T$

- The **Advantage Estimate** $\hat{A}_t$ quantifies how much better an action is compared to the expected outcome.
- To compute $\hat{A}_t$ we need:

- **Discounted sum of rewards:**

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

- **Estimated discounted reward from this point onward (Value Function Estimate $V(s_t)$):**

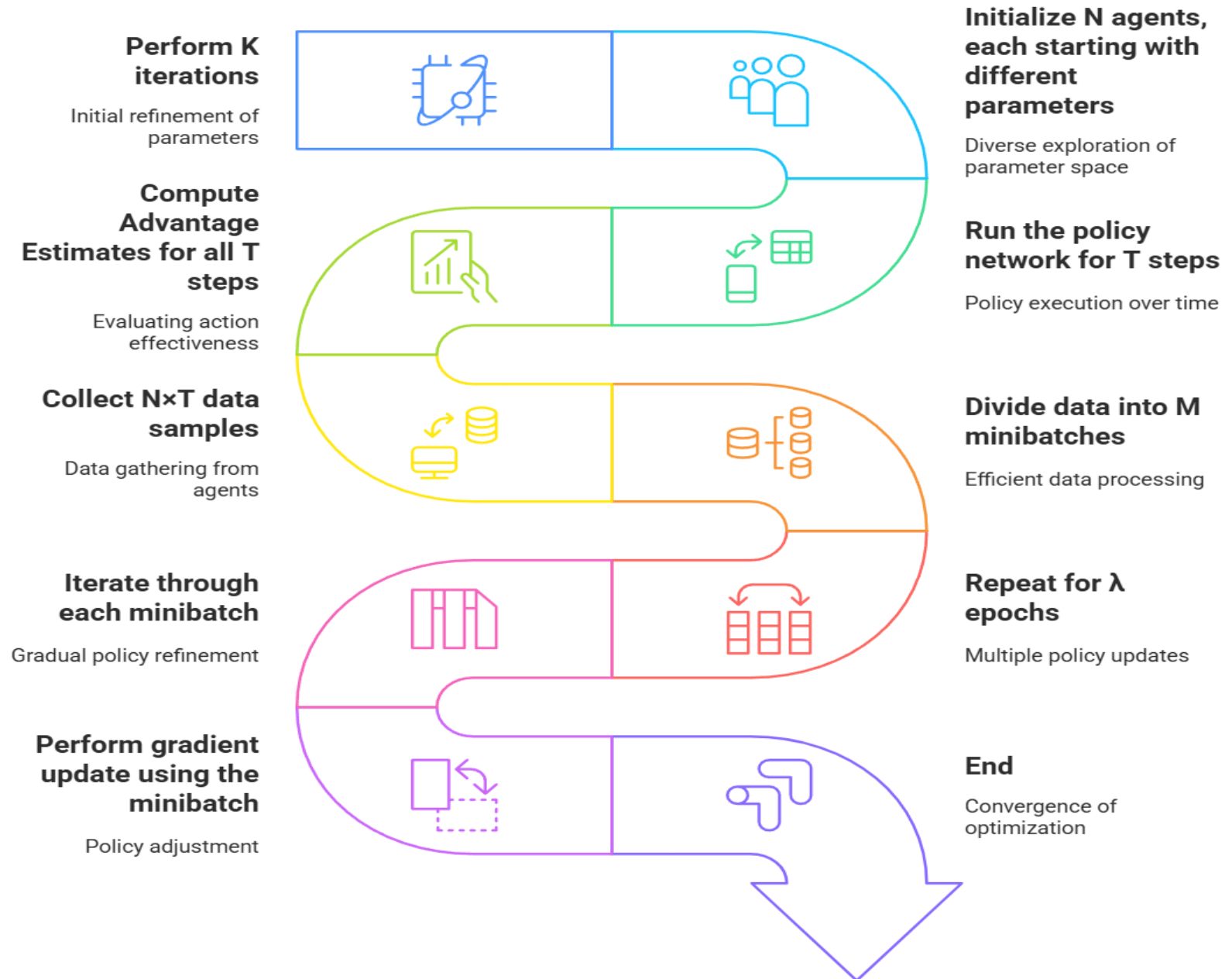
$$\hat{A}_t = G_t - V(s_t)$$

Training Process

- The **Value Function Network** provides $V(s_t)$, an estimate of expected future rewards.
 - The advantage estimate tells us whether an action is better or worse than expected.
 - 5 **Collect $N \times T$ data samples**
 - Gather experiences from all **N agents** over **T timesteps** to create a dataset of **state-action-reward transitions**.
 - 6 **Divide the $N \times T$ data into M minibatches ($M \leq N \times T$)**
 - To improve training stability, the collected data is divided into **M minibatches**, ensuring efficient updates.
 - 7 **Repeat for λ epochs:**
 - The policy is updated multiple times using the collected data.
 - 8 **Iterate through each minibatch**
 - Each minibatch is used for gradient updates to refine the policy network.
 - 9 **Perform gradient update using the minibatch**
 - The **clipped surrogate objective** is used to ensure stable and constrained policy updates:
- End**

Iterative Optimization of Quantum Circuit Parameters

Work flow



Loss Function

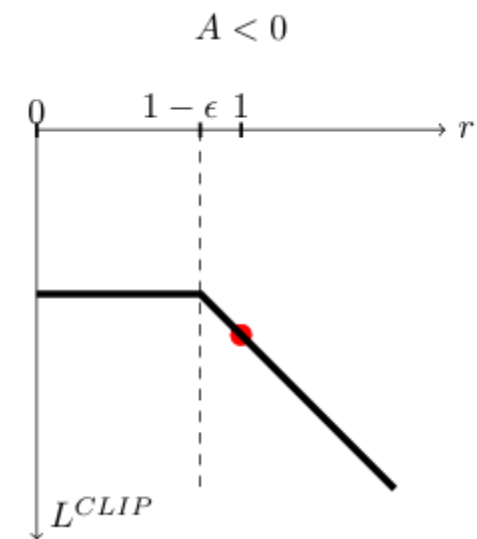
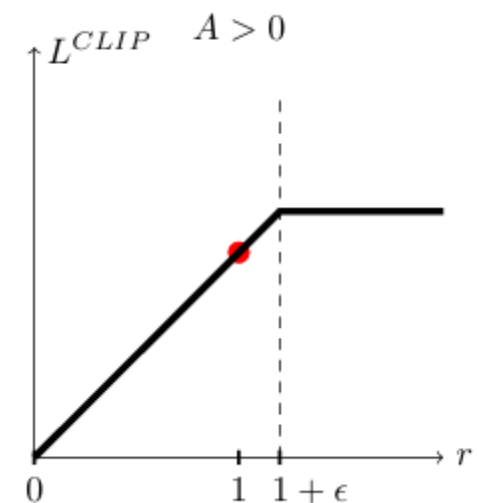
Clipped Surrogate Objective (Policy Loss)

The policy loss function in PPO ensures stable learning by limiting the size of policy updates:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

where:

- $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ is the **probability ratio** between new and old policies.
- $\hat{A}_t$ is the **advantage estimate**.
- ϵ is a **clipping parameter** (e.g., 0.2) that prevents large updates.



Loss Function

Value Loss (Critic Loss)

The value network minimizes the **mean squared error** (MSE) between the predicted value function and the actual return G_t :

$$L^{VF}(\theta) = (V_{(s_t)} - G_t)^2 \text{ where:}$$

- $V_{(s_t)}$ is the **estimated value function**.
- G_t is the **discounted return** from state s_t .

Entropy Bonus (Optional, for Exploration)

To encourage exploration, an **entropy term** is added to the policy loss:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) + c S[\pi_\theta](s_t) \right]$$

where:

- $S[\pi_\theta](s_t)$ is the **entropy** of the policy at state s_t .
- c is a **hyperparameter** controlling the strength of the entropy bonus.

Combined PPO Loss

The final loss function combines **policy loss, value loss, and entropy bonus**:

$$L = L^{CLIP} - c_1 L^{VF} + c_2 S$$

where:

- c_1 is the **coefficient of the value loss**.
- c_2 is the **entropy coefficient**.

Exploration Strategy

Stochastic Policy:

- The policy outputs a probability distribution over actions, ensuring natural exploration by sampling different actions.

Entropy Regularization:

- An entropy bonus can be added to encourage the policy to be more random, promoting better exploration.

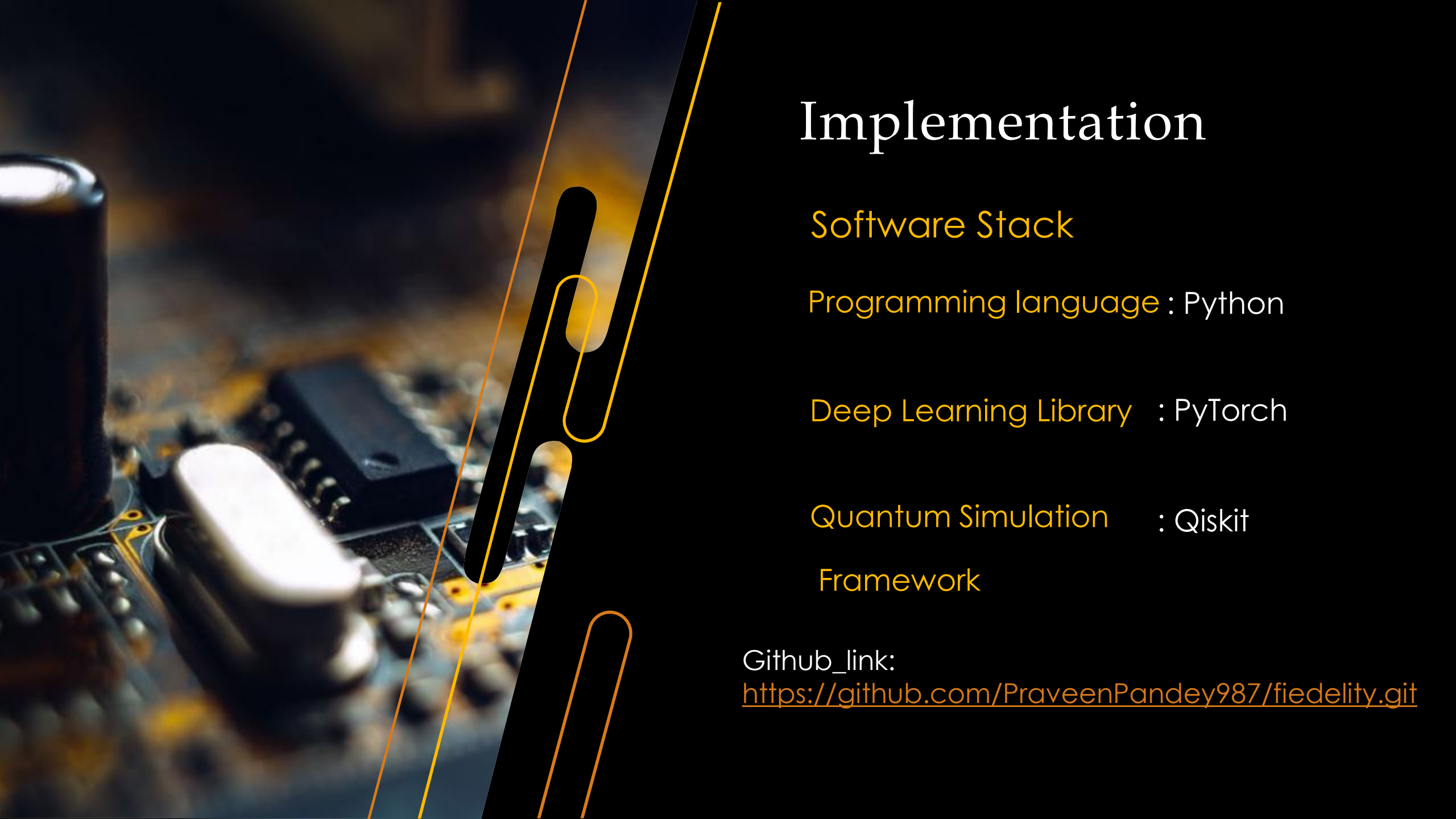
Action Clipping:

- Actions are clipped within a bounded range to prevent excessively large parameter updates, improving stability and controlled exploration.

Parameter Noise (Alternative Approach):

- Instead of (or in addition to) using a stochastic policy, noise can be directly added to the parameters during training to induce exploration.

Hyperparameter	Value (Example)	Description
Learning Rate (Policy)	3e-4	Learning rate for the policy network optimizer.
Learning Rate (Value)	1e-3	Learning rate for the value network optimizer.
Discount Factor (γ)	0.99	Discount factor for future rewards.
Clipping Range (ϵ)	0.2	Clipping parameter for the PPO surrogate objective.
Hidden Layer Size	64	Number of neurons in each hidden layer of the networks.
Number of Rollouts	10	Number of trajectories to collect per iteration.
Rollout Length	20	Number of steps per trajectory.
Number of Epochs	5	Number of times to update the networks per iteration.
Entropy Bonus Coefficient	0.01	Strength of the entropy regularization term (if used).
GAE Parameter (λ)	0.95	Parameter for Generalized Advantage Estimation.
Optimizer	Adam	Optimizer for the neural network.
Action Bound	0.1	Maximum absolute value the parameters can be updated in one iteration.



Implementation

Software Stack

Programming language : Python

Deep Learning Library : PyTorch

Quantum Simulation : Qiskit

Framework

Github_link:

<https://github.com/PraveenPandey987/fidelity.git>

References

- 1. <https://arxiv.org/abs/1707.06347>
- 2. <https://www.davidsilver.uk/teaching/>
- 3. <https://sites.google.com/view/deep-rl-bootcamp/lectures>
- 4. https://github.com/yandexdataschool/Practical_RL/tree/master/week06_policy_based
- 5. <https://disk.yandex.com/i/flW8ezGk3TPsQ5>





Thank you